

021 Trade Hackathon – API Guide

Quick start

You can go from sign-up to a live order and streaming prices in about 15 minutes. Every account is a virtual trading account with ₹10,00,000 of funds on NSE and BSE (cash and F&O).

1. **Create an account** in the web app and verify your mobile number (section *Create your account*).
2. **Note your UCC** (client code, looks like HACK1234) from your Profile. This is your API username.
3. **Get an access token:** POST /auth/token with your UCC and the password you chose at sign-up.
4. **Call the REST APIs** with Authorization: Bearer <accessToken> : download instruments, place orders, read positions.
5. **Stream data:** get an ephemeral key from GET /websocket/ephemeral-key , then open the market and/or orders websocket with ?token=<key> .

What	URL
Web app (sign-up, profile)	https://devcarbon.021.trade/auth
REST base URL	https://devapi.021.trade/api/developer-api/v1
Market data websocket	wss://devapi.021.trade/api/developer/websocket/market?token=<ephemeral key>
Order updates websocket	wss://devapi.021.trade/api/developer/websocket/orders?token=<ephemeral key>

Problem statement rules

Two problem statements have extra rules on which APIs you may use and what you must build. Read the one you picked.

021-02: Paper trading engine (021-02-paper-trading-engine.md)

Build your own paper-trading engine on live market data. **Do not use the order APIs:** orders placed by your app's users must stay inside your application and must never be sent to the broker.

Allowed	Not allowed
POST /auth/token	POST /orders , PUT /orders/{id} , DELETE /orders/{id}
GET /instruments	GET /orders , GET /orders/{id} , GET /trades , GET /orders/{id}/trades
GET /websocket/ephemeral-key	GET / PUT /portfolio/positions , GET /portfolio/holdings
Market data websocket	Orders websocket

Your engine owns the whole order lifecycle: it accepts users' orders, fills them against the live prices from the market data websocket, and keeps its own order book, trades, positions, funds and P&L.

021-03: Algo trading platform (021-03-algo-trading-platform.md)

Your platform must run these **two strategies** below, plus at least one strategy of your own, end to end with the APIs in this guide. Prices come from the market data websocket; orders go through POST /orders .

Strategy 1: time-based entry and exit

1. At **9:15 AM IST**, enter the position: place the order for your chosen instrument, side and quantity, with product: INTRADAY .
2. At **3:15 PM IST**, exit: square off the full position with an opposite order (use squareOffQuantity from GET /portfolio/positions), and cancel any of its orders that are still open.

Strategy 2: 1% breakout from the open, with 5% target and 5% stop-loss

1. Subscribe the stock on the market socket and take today's **open price**: the open field of the full packet, or the open in the ltpo snapshot.
2. On every LTP update, compare it with the open:
 - $LTP \geq open \times 1.01$ (up 1% or more): **buy**.
 - $LTP \leq open \times 0.99$ (down 1% or more): **sell** (qty negative).

3. Once the entry fills, **your platform manages the 5% target and 5% stop-loss itself**. Keep watching LTP, and when either level is hit, close the position with an opposite order of the same quantity. Don't use any built-in target or stop-loss feature; the API doesn't take these levels.

Entry side	Target	Stop-loss
Buy at price P	$P \times 1.05$	$P \times 0.95$
Sell at price P	$P \times 0.95$	$P \times 1.05$

Example: the open is ₹1,400.00 and LTP reaches ₹1,414.00 (+1%), so you buy 1 share at 141400 paise. Your platform then exits with a sell order when LTP reaches the target, $141400 \times 1.05 = 148470$, or the stop-loss, $141400 \times 0.95 = 134330$ (both rounded to the tick size).

```
{"exchange": "NSE", "token": 2885, "qty": 1, "price": 141400, "book": "RL", "product": "INTRADAY", "validity": "Day"}
```

The exit when the target or stop-loss is hit is the same order with `"qty": -1` at the exit price. Track fills with the orders websocket and `GET /orders`, and your open quantity with `GET /portfolio/positions`.

Strategy 3: your own (mandatory)

Add at least one strategy you design yourself. It doesn't need to be profitable, but it must follow clear, written rules: what it trades, when it enters, how it sizes the position, and when it exits (target, stop-loss or time). For example, "buy when the 5-minute average crosses above the 20-minute average, exit on the opposite cross or at 3:15 PM." Write the rules down in your submission so the judges can check that the platform follows them.

Create your account

Sign up once in the web app. Your account is fully set up (KYC done, funds added, all segments on) the moment you verify your mobile OTP.

1. Open <https://devcarbon.021.trade/auth>. On **Let's get started**, enter your email and click **Continue**.
2. On **Create your password**, choose a password and click **Continue**. Remember it: the API login uses this password.
3. On **Verify your email**, open the verification link sent to your inbox (check spam too). Then click **Refresh** on the page. Use **Resend** if the mail doesn't arrive.

4. On **Enter your mobile number**, type your 10-digit number (+91) and click **Generate OTP**.
5. On **Enter your OTP**, type the 6-digit code from the SMS. Your account is now activated automatically.
6. In the app, open **Profile** and note the **UCC** field (format `HACK<number>`, e.g. `HACK1234`). This is your **API username**.

What you get:

- ₹10,00,000 virtual funds, topped back up to ₹10,00,000 at the start of every trading day.
- NSE and BSE, cash and F&O, all enabled.
- Orders go to a simulated exchange, not the live market.

Use one email and one mobile number per participant. If you see a KYC or onboarding screen after the OTP, sign out and sign in again; if it persists, contact the organisers.

Basics

All REST calls go to `https://devapi.021.trade/api/developer-api/v1`, send JSON, and (except login) need the header `Authorization: Bearer <accessToken>`.

Units, the #1 source of mistakes

- **Prices are integers in paise.** ₹1 = 100, so ₹1,405.50 is sent as `140550`. This applies to price, trigger, circuits, strikes and tick size.
- **Quantity sign is the side.** Positive `qty` = buy, negative `qty` = sell.
- **Quantity is in units, not lots.** For F&O it must be a multiple of the lot size (`board_lot_quantity` in the instruments file).
- **token** is the instrument id from the instruments file (`GET /instruments`).

Exchange names. Requests use the short names on the left; responses and the instruments file use the internal names.

Request value	Response / instruments value	Websocket code	Segment
NSE	NSECM	1	NSE cash (equity)
NSEFO	NSEFO	2	NSE F&O
–	NSEIDX	3	NSE indices (data only, not tradable)

Request value	Response / instruments value	Websocket code	Segment
BSE	BSEEQ	4	BSE cash (equity)
BSEFO	BSEEQD	5	BSE F&O
–	BSEIDX	6	BSE indices (data only, not tradable)

Order fields with fixed values

Field	Values	Meaning
product	INTRADAY , CNC , NRML	INTRADAY: any segment, squared off same day. CNC: cash delivery only. NRML: F&O carry-forward only.
book	RL , SL	RL: regular limit order. SL: stop-loss order (needs trigger).
validity	Day , IOC	Day: stays open till market close. IOC: fills immediately or is cancelled.
status (responses)	Received , Placed , Pending , Executed , Rejected , Cancelled , Frozen , SentForModification , SentForCancellation , AcceptedForAMO	Order lifecycle state. Rejected orders carry the reason in reason .

Response format. Login, order actions and all errors return an envelope; list endpoints return a plain JSON array.

```
{ "data": { ... }, "success": true, "error": null }
{ "data": null, "success": false, "error": "Invalid or expired access token."
}
```

HTTP status	When
200	Success
400	Bad body, unknown exchange/product/book/validity, invalid order id

HTTP status	When
401	Missing/expired token, wrong credentials, account not allowed to trade
403	Account inactive, or not a client account
404	Order id not found (GET /orders/{id})
421	Account belongs to another cluster (contact organisers)
422	Order blocked by safe mode (the error text says why)
500	Order rejected by risk checks, or a server error; read error
503	Service temporarily unavailable; retry after a few seconds

Authentication

Exchange your UCC and password for an access token once a day, then send it on every call. The token is valid until **5:00 AM IST the next morning**.

POST /auth/token (no Authorization header)

```
curl -X POST "https://devapi.021.trade/api/developer-api/v1/auth/token" \
-H "Content-Type: application/json" \
-d '{"username":"HACK1234","password":"<your password>"}'
```

Request field	Meaning
username	Your UCC from Profile (e.g. HACK1234). Not your email.
password	The password you created at sign-up.

```
{
  "data": {
    "accessToken": "8f2c...e91a",
    "tokenType": "Bearer",
    "expiresAt": "2026-10-09T05:00:00+05:30",
    "expiresIn": 39600
  },
  "success": true,
  "error": null
}
```

Response field	Meaning
accessToken	Send as <code>Authorization: Bearer <accessToken></code> on every other call.
tokenType	Always <code>Bearer</code> .
expiresAt	When the token stops working (next 5 AM IST), RFC 3339.
expiresIn	Seconds left until <code>expiresAt</code> .

Rules to know:

- **One live token per account.** Logging in again revokes your previous token immediately. Log in once and share the token across your scripts.
- There is no refresh token. After expiry (or a 401 `Invalid or expired access token` .), just log in again.
- Wrong UCC or password both return 401 `Invalid username or password` .

Instruments

Download the full instrument list (about 1,05,000 rows, NSE + BSE) once a day and look up the `token` you need for orders and websockets.

GET /instruments returns a gzipped CSV. Use `--compressed` with curl (Postman and Python `requests` decompress automatically).

```
curl --compressed "https://devapi.021.trade/api/developer-api/v1/instruments" \
  -H "Authorization: Bearer <accessToken>" -o instruments.csv
```

```
token,exchange,underlying_token,underlying_exchange,min_lot_size,board_lot_quantity,freeze_quantity,lower_circuit,upper_circuit,expiry,option_type,strike_price,symbol,instrument_type,isin,ticksiz
22699,NSECM,22699,NSECM,0,100,984600,9649,10663,0,,0,747HR36,STK,IN1620230376,1
```

Column	Meaning
token	Instrument id; use it in orders and websocket subscriptions. Unique per exchange.
exchange	NSECM , NSEFO , NSEIDX , BSEEQ , BSEEQD , BSEIDX (see Basics for request names).
underlying_token	Token of the underlying (for F&O); same as token for cash instruments.
underlying_exchange	Exchange of the underlying.
min_lot_size	Minimum lot size as provided by the exchange (may be 0).
board_lot_quantity	Lot size: order quantity must be a multiple of this.
freeze_quantity	Maximum quantity allowed in a single order.
lower_circuit	Lowest price allowed today, in paise.
upper_circuit	Highest price allowed today, in paise.
expiry	Contract expiry as an integer timestamp from the exchange; 0 for cash instruments. Larger = later expiry.
option_type	CALL , PUT , or empty for non-options.
strike_price	Option strike in paise; 0 for non-options.
symbol	Trading symbol (e.g. RELIANCE , NIFTY).
instrument_type	STK stock, IDX index, FUTIDX / FUTSTK index/stock future, OPTIDX / OPTSTK index/stock option.
isin	ISIN of the security (blank for derivatives).
ticksiz	Minimum price step in paise: prices must be a multiple of this.

Tips: filter by `exchange` + `symbol` + `instrument_type` to find a contract; for options also match `expiry`, `strike_price` and `option_type`. The list refreshes daily before market open.

Orders

Place, modify and cancel orders, and read today's order book. Every order passes the same risk checks as the 021 Trade app (funds, circuits, lot size, tick size, trigger rules).

Place an order: `POST /orders`

```
curl -X POST "https://devapi.021.trade/api/developer-api/v1/orders" \
  -H "Authorization: Bearer <accessToken>" -H "Content-Type:
application/json" \
  -d
'{"exchange":"NSE","token":2885,"qty":1,"price":140000,"book":"RL","product":
"INTRADAY","validity":"Day"}'
```

Field	Required	Meaning
<code>exchange</code>	Yes	<code>NSE</code> , <code>NSEFO</code> , <code>BSE</code> or <code>BSEFO</code> .
<code>token</code>	Yes	Instrument token from <code>GET /instruments</code> .
<code>qty</code>	Yes	Quantity in units; positive = buy, negative = sell; F&O in multiples of the lot size.
<code>price</code>	Yes	Limit price in paise. <code>0</code> = market order (fills at the best available price).
<code>book</code>	Yes	<code>RL</code> regular or <code>SL</code> stop-loss.
<code>product</code>	Yes	<code>INTRADAY</code> , <code>CNC</code> (cash only) or <code>NRML</code> (F&O only).
<code>validity</code>	Yes	<code>Day</code> or <code>IOC</code> .
<code>trigger</code>	For <code>SL</code>	Trigger price in paise; the order activates when LTP crosses it.
<code>discQuantity</code>	No	Disclosed quantity: only this much is shown in the market depth at a time.
<code>amo</code>	No	<code>true</code> = after-market order, queued now and sent at the next market open.

Response:

```
{ "data": { "type": "SUCCESS", "message": "Order placed successfully",
"orderId": "1042" }, "success": true, "error": null }
```

Field	Meaning
orderId	Your order id (string). Use it to modify, cancel or look up the order.
message	Human-readable result from the risk system.

A risk rejection (low funds, price outside circuit, wrong lot size...) returns `success: false` with the reason in `error`.

Modify an order: `PUT /orders/{orderId}`

Send the **full new state** of the order (not just the changed fields). Only pending orders can be modified.

```
curl -X PUT "https://devapi.021.trade/api/developer-api/v1/orders/1042" \
-H "Authorization: Bearer <accessToken>" -H "Content-Type:
application/json" \
-d
'{"exchange":"NSE","token":2885,"qty":1,"price":139500,"book":"RL","product":
"INTRADAY","validity":"Day"}'
```

Fields are the same as place order: `exchange`, `token`, `qty`, `price`, `book`, `product`, `validity`, and `trigger` / `discQuantity` / `amo` where used.

Response: `{"data":`

`{"type":"SUCCESS","message":"..."},"success":true,"error":null}`.

Cancel an order: `DELETE /orders/{orderId}`

```
curl -X DELETE "https://devapi.021.trade/api/developer-api/v1/orders/1042" \
-H "Authorization: Bearer <accessToken>" -H "Content-Type:
application/json" \
-d '{"exchange":"NSE","token":2885,"product":"INTRADAY"}'
```

Field	Meaning
exchange	Exchange of the order: <code>NSE</code> , <code>NSEFO</code> , <code>BSE</code> or <code>BSEFO</code> .
token	Instrument token of the order.

Field	Meaning
product	Product of the order.

Response: same envelope as modify. A cancel request is asynchronous: the order shows `SentForCancellation` and then `Cancelled`.

List today's orders: `GET /orders`

Returns a plain JSON array of all today's orders (open and completed), newest first. `GET /orders/{orderId}` returns the same object for one order inside a one-element array, or 404.

```
[{"orderId":1042,"time":1791451200,"lastActivity":1791451201,"token":2885,"exchange":"NSECM","product":"INTRADAY",
"qtyRemaining":0,"qtyTraded":1,"discQty":0,"price":140000,"triggerPrice":0,"book":"RL","validity":"Day",
"status":"Executed","reason":"","placedBy":"User","amo":false,"exchangeOrderNumber":"1100000012345678"}]
```

Field	Meaning
orderId	Your order id (number here; the same id returned by place order).
time	When the order was placed, Unix seconds.
lastActivity	Time of the latest exchange update on the order, in seconds.
token, exchange, product	Instrument and product of the order.
qtyRemaining	Quantity still open; positive = buy, negative = sell.
qtyTraded	Quantity filled so far.
discQty	Disclosed quantity.
price	Limit price in paise (0 = market).
triggerPrice	Stop-loss trigger in paise (0 if none).
book, validity, amo	As sent when placing.
status	Current state (see Basics).

Field	Meaning
reason	Rejection or cancellation reason, if any.
placedBy	User (you) or RMS (automatic square-off).
exchangeOrderNumber	Order number at the exchange (empty until accepted).

Responses may contain a few more fields than listed here; ignore them.

Trades

Each fill (full or partial) of an order is a trade. Both endpoints return a plain JSON array.

- **GET /trades** : all of today's trades.
- **GET /orders/{orderId}/trades** : trades of one order; an empty array if it has no fills yet (or the id isn't yours).

```
curl "https://devapi.021.trade/api/developer-api/v1/trades" -H
"Authorization: Bearer <accessToken>"
```

```
[{"tradeId":5001,"tradeTime":1791451201,"quantity":1,"token":2885,"exchange":
"NSECM","product":"INTRADAY","price":140000,"symbol":"RELIANCE"}]
```

Field	Meaning
tradeId	Trade id at the exchange.
tradeTime	When the fill happened, in seconds.
quantity	Filled quantity; positive = bought, negative = sold.
token , exchange , product	Instrument and product.
price	Fill price in paise.
symbol	Trading symbol.

Portfolio

Positions are what you traded today (plus F&O carried forward); holdings are delivery shares in your demat. INTRADAY positions are squared off automatically before market close.

Positions: GET /portfolio/positions

```
[{"exchange":"NSECM","token":2885,"product":"INTRADAY","netQuantity":10,"netPrice":140000,"carryForwardQuantity":0,"longPosition":true,"squareOffQuantity":-10,"buyPrice":140000,"sellPrice":0,"prevClose":139200}]
```

Field	Meaning
exchange , token , product	Identify the position (one row per instrument + product).
netQuantity	Open quantity now; positive = long, negative = short, 0 = closed.
netPrice	Average price of the open quantity, in paise.
carryForwardQuantity	Quantity brought in from previous days (NRML).
longPosition	true if the position was opened by buying.
squareOffQuantity	Quantity to send (with this sign) to close the position.
buyPrice	Average buy price today, in paise.
sellPrice	Average sell price today, in paise.
prevClose	Previous close in paise; P&L = (LTP – netPrice) × netQuantity / 100 in ₹.

Responses may contain a few more fields than listed here; ignore them.

Convert product: PUT /portfolio/positions

Moves an open position between products, e.g. INTRADAY to CNC to hold it overnight.

You will most likely never need this API for the hackathon; it is listed only for completeness.

```
curl -X PUT "https://devapi.021.trade/api/developer-api/v1/portfolio/positions" \
  -H "Authorization: Bearer <accessToken>" -H "Content-Type: application/json" \
  -d '{"exchange":"NSE","token":2885,"oldProduct":"INTRADAY","newProduct":"CNC","quantity":10}'
```

Field	Meaning
exchange	NSE , NSEFO , BSE or BSEFO .
token	Instrument token of the position.
oldProduct	Current product of the position.
newProduct	Product to convert to (CNC only for cash, NRML only for F&O).
qty	Quantity to convert, with the position's sign (positive for a long position).

Response: {"data":

{"type":"SUCCESS","message":"..."},"success":true,"error":null}.

Holdings: GET /portfolio/holdings

```
[{"isin":"INE002A01018","symbol":"RELIANCE","freeQty":10,"btstQty":0,"sellQty":0,"price":138000,"prevClose":139200}]
```

Field	Meaning
isin	ISIN of the share.
symbol	Trading symbol.
freeQty	Settled quantity you can sell.
btstQty	Bought recently, not yet settled (T+1); can still be sold.
sellQty	Quantity already sold or in pending sell orders today.
price	Average buy price in paise.
prevClose	Previous close in paise.

To sell a holding, place a sell order (qty negative) with product: CNC on NSE or BSE using the token for that ISIN.

Websockets

Use the **market** socket for live prices and the **orders** socket for real-time updates on your own orders and trades. Both need an ephemeral key, not your access token.

Socket	URL	Use it for	You send
Market data	<code>wss://devapi.021.trade/api/developer/websocket/market?token=<key></code>	LTP, full depth/OHLC/OI, option-chain greeks	Subscribe / unsubscribe JSON
Order updates	<code>wss://devapi.021.trade/api/developer/websocket/orders?token=<key></code>	Order accepted/rejected/modified/cancelled, trades, SL triggered	Nothing

Step 1: get an ephemeral key: `GET /websocket/ephemeral-key`

```
curl "https://devapi.021.trade/api/developer-api/v1/websocket/ephemeral-key"
-H "Authorization: Bearer <accessToken>"
```

```
{ "data": { "token": "eyJhbGciOiJIUzI1NiIs..."}, "success": true, "error": null }
```

`data.token` is the key: pass it as `?token=` when connecting. It is valid for 24 hours and is only checked when the socket connects, so an open socket keeps working after it expires. Get a fresh key whenever you reconnect.

Rules for both sockets

- **Binary, big-endian.** Server messages are binary frames; every integer is big-endian; prices are in paise.
- **Every packet starts with a 2-byte transaction code (TC)** that tells you its layout.
- **Keep-alive is automatic.** The server pings every 20 s and drops you after 30 s of silence; standard websocket clients answer pings for you. Ignore the 2-byte frame `[0x00, 0x0A]` (TC 10, heartbeat).
- **Limits:** 20 open connections per socket type per account; 100 instruments per market connection; client messages up to 5,000 bytes (larger closes the socket).
- **Bad or expired key:** the handshake fails with HTTP 403.
- **Errors are silent.** An invalid subscription is ignored with no reply; the socket stays open.
- **Daily restart at 08:00 IST.** All sockets drop. Reconnect with backoff, get a new key and resubscribe; subscriptions are not remembered.

Market socket: subscribe

Send a **text** frame with JSON:

```
{"Task": "subscribe", "Mode": "ltp", "Instruments": [[1, 2885], [2, 43210]]}
```

Field	Meaning
Task	subscribe or unsubscribe (lowercase).
Mode	ltp last price · ltpo LTP + open · ltpc LTP + previous close · full full packet (depth, OHLC, volume, OI) · oc option-chain data for options.
Instruments	List of [exchangeCode, token]; exchange codes 1 NSECM, 2 NSEFO, 3 NSEIDX, 4 BSEEQ, 5 BSEEQD, 6 BSEIDX.
Filters	Only for oc: comma-separated list of l LTP, o OI, v volume, i IV, md best bid/ask, g greeks, poi PCR-OI, e.g. "l,o,v,i,md,g,poi".

How data arrives:

- Right after every subscribe/unsubscribe you get a **snapshot** frame with the current data of all your subscribed instruments.
- After that, updates come at most every **300 ms**, containing only instruments that changed. Values are the latest snapshot, not every tick.
- One frame holds **many packets back to back**: read the TC, read that packet's length from the tables below, move on.
- Each instrument has one mode per connection; subscribing again changes its mode. Unknown tokens are dropped silently.
- To unsubscribe, send the same JSON with "Task": "unsubscribe" and any valid Mode (oc for option-chain subscriptions, any other for the rest).

TC 1: LTP (modes ltp, ltpo, ltpc), 12 bytes

Offset	Size	Field
0	2	TC = 1
2	2	Exchange code (1-6)
4	4	Token

Offset	Size	Field
8	4	LTP (paise)

In **snapshot** frames only, `ltpo` and `ltpc` packets carry 5 extra bytes: 1 byte `'o'` (0x6F) or `'c'` (0x63), then a 4-byte open or previous-close price. Detect it by peeking at the next byte: a new packet always starts with 0x00.

TC 3: Full (mode `full`). Layout depends on the exchange code at offset 2.

Exchange	Length	Layout (offset: field, size in bytes)
1 NSECM	220	0 TC(2) · 2 exch(2) · 4 token(4) · 8 LTP(4) · 12 last trade qty(4) · 16 avg price(4) · 20 volume(8) · 28 total buy qty(8) · 36 total sell qty(8) · 44 close(4) · 48 open(4) · 52 high(4) · 56 low(4) · 60 last trade time(4) · 64 depth(140) · 204 indicative close(4) · 208 indicative imbalance qty(8) · 216 exchange time(4)
2 NSEFO	234	0 TC(2) · 2 exch(2) · 4 packet id(2) · 6 token(4) · 10 LTP(4) · 14 last trade qty(4) · 18 avg price(4) · 22 volume(8) · 30 total buy qty(8) · 38 total sell qty(8) · 46 close, open, high, low(4 each) · 62 last trade time(4) · 66 depth(140) · 206 OI(8) · 214 OI high(8) · 222 OI low(8) · 230 exchange time(4)
3 NSEIDX	36	0 TC(2) · 2 exch(2) · 4 token(4) · 8 index value(4) · 12 close, open, high, low(4 each) · 28 exchange time(4) · 32 indicative close(4)
4/5/6 BSE	262	0 TC(2) · 2 exch(2) · 4 token(4) · 8 LTP(4) · 12 last trade qty(8) · 20 avg price(4) · 24 volume(8) · 32 total buy qty(8) · 40 total sell qty(8) · 48 close, open, high, low(4 each) · 64 last trade time(4) · 68 depth(160) · 228 OI(8) · 236 OI high(4) · 240 OI low(4) · 244 exchange time(4) · 248 auction/close data(14)

- **Depth** = 10 levels, first 5 bids (best first) then 5 asks. NSE level = qty(8) + price(4) + orders(2) = 14 bytes; BSE level = qty(8) + price(4) + orders(4) = 16 bytes.
- Times are seconds since 1980-01-01 00:00:00 (add 315513000 for Unix seconds in IST-based exchange time).

TC 9: Option chain (mode `oc`, NSEFO/BSEEQD options only)

Offset	Size	Field
0	2	TC = 9
2	4	Token
6	2	Exchange code
8	1	Number of fields that follow
9	...	One entry per filter, in the order you sent: 1-byte id + value

Id	Filter	Value
1	<code>l</code>	LTP, 4 bytes, paise
2	<code>o</code>	OI, 8 bytes
3	<code>v</code>	Volume, 8 bytes
4	<code>i</code>	IV, 4 bytes, $\times 10,000$ (2534 = 25.34%)
7	<code>md</code>	32 bytes: best bid price, best ask price, bid qty, ask qty (8 bytes each; prices in paise)
8	<code>g</code>	20 bytes: delta, gamma, vega, theta, rho (4 bytes each, $\times 10,000$)
9	<code>poi</code>	PCR by OI, 4 bytes, $\times 100$

Orders socket

Connect and listen; you don't need to send anything. You receive only **your own** order events, one packet per binary frame, in order. Events that happen while you are disconnected are **not replayed**: after reconnecting, call `GET /orders` and `GET /trades` to catch up.

Status values (offset 2 in both layouts):

Value	Event
1	Trade (order filled, fully or partly)
2	New order accepted by the exchange
3	Order rejected

Value	Event
4	Stop-loss triggered
5	Modify accepted
6	Modify rejected
7	Cancelled
8	Cancel rejected
9	Price confirmation

TC 4: NSE order event, 46 bytes + optional text

Offset	Size	Field
0	2	TC = 4
2	2	Status (table above)
4	10	Your UCC, ASCII, space-padded
14	20	Symbol, ASCII (first 10 bytes; rest zero)
34	4	Order id (same as <code>orderId</code> in REST)
38	4	Quantity; negative = sell (fill quantity for trades)
42	4	Price in paise (fill price for trades)
46	rest	Rejection reason text, ASCII, only on errors

TC 8: BSE order event, 32 bytes + optional text

Offset	Size	Field
0	2	TC = 8
2	2	Status (table above)
4	10	Your UCC, ASCII, space-padded
14	2	Exchange code (4 BSEEQ, 5 BSEEQD)
16	4	Token
20	4	Order id (same as <code>orderId</code> in REST)
24	4	Quantity; negative = sell (fill quantity for trades)

Offset	Size	Field
28	4	Price (fill price for trades); confirm exact values with <code>GET /orders</code>
32	rest	Reason text, ASCII, for rejections and cancellations

Use the order id to match events to your orders; for NSE F&O the symbol field can be blank, so rely on the order id.

Common mistakes and FAQ

Symptom	Cause and fix
Login returns 401	Use your UCC (<code>HACK...</code>) as username, not your email; check the password from sign-up.
A working token suddenly gives 401	You (or another script) logged in again, which revokes older tokens, or it is past 5 AM IST. Log in once and reuse the token.
Order rejected for price or quantity	Price must be in paise and a multiple of <code>ticksize</code> , within the circuit limits; F&O quantity must be a multiple of <code>board_lot_quantity</code> .
<code>CNC</code> is only valid for equity instruments / <code>NRML</code> is only valid for F&O instruments	Use <code>CNC</code> only with <code>NSE / BSE</code> , <code>NRML</code> only with <code>NSEFO / BSEFO</code> ; <code>INTRADAY</code> works everywhere.
SL order rejected	Buy SL: trigger above LTP and price $\geq$ trigger. Sell SL: trigger below LTP and price $\leq$ trigger.
Insufficient funds	Funds are ₹10 lakh per day; open positions and pending orders block margin. Square off or cancel to free it.
Websocket handshake fails with 403	The ephemeral key is missing, malformed or older than 24 h; fetch a new one.
Subscribed but no data	Wrong exchange code (websockets use numbers 1-6), unknown token, or market closed (you only get the last snapshot).

Symptom	Cause and fix
Can't unsubscribe when at 100 instruments	The limit counts current + requested instruments, so unsubscribe in small batches, or open another connection.
oc subscription stops all data on that connection	A filter code was invalid. Use only <code>l,o,v,i,md,g,poi</code> .
Missed order events	The orders socket doesn't replay; call <code>GET /orders</code> and <code>GET /trades</code> after reconnecting.
Sockets dropped at 8 AM	Daily restart. Reconnect with backoff, fetch a new key, resubscribe.

Good practice:

- Download instruments once a day and cache them; don't fetch per order.
- Keep one access token per account and one market + one orders connection; subscribe many instruments on one connection.
- Treat REST (`GET /orders` , `/trades` , `/portfolio/positions`) as the source of truth and websockets as live updates.
- Prefer **NSE** instruments and orders over BSE. It keeps things simple for you and makes testing easier for us.
- Skipping a feature or part of the scope is perfectly fine and won't be judged harshly, as long as you list it as future scope in your README.
- Everything is virtual money on a simulated exchange: experiment freely.

Need help? Contact the organisers at *<support channel / email>* with your UCC, the request you sent and the response you got.